

Detector de Risco em Componentes de IA de Terceiros para Ambientes Críticos

Integrantes: Kayron Gabriel Gomes Nascimento, Lucas Alves Santana

1 Introdução

Atualmente, muitos sistemas utilizam componentes de IA de terceiros, como modelos, “datasets” (ou conjunto de dados), “APIs” (Interface de Programação de Aplicações) e bibliotecas, sem uma análise adequada de risco antes da integração. Isso pode gerar problemas de segurança, incompatibilidade, uso de componentes sem procedência confiável, falhas de integridade e até questões legais relacionadas à licença. Diante disso, o projeto propõe uma solução capaz de avaliar esses componentes antes que sejam usados em sistemas críticos.

Esse projeto busca resolver um problema atual da área de segurança digital: o uso de componentes de IA de terceiros sem uma validação prévia de risco. A proposta é criar um sistema que analise esses componentes e informe se eles devem ser aprovados, revisados ou bloqueados.

2 Tema

Segurança no uso de IA de terceiros.

3 Contexto

Com o crescimento do uso de inteligência artificial, organizações passaram a integrar rapidamente bibliotecas, APIs, “datasets” e modelos desenvolvidos por terceiros. Porém, essa adoção nem sempre vem acompanhada de uma verificação de procedência, integridade, licença, atualização ou dependências. Em ambientes críticos, como sistemas públicos, corporativos ou de apoio à decisão, isso representa um risco relevante.

Falta uma camada simples e objetiva para avaliar o risco de componentes de IA antes da integração em um sistema. Assim, tem-se como proposta desenvolver um sistema que receba informações sobre um componente de IA e gere uma classificação de risco baseada em regras de segurança e conformidade.

4 Levantamento e análise de requisitos

Nos requisitos funcionais, define-se tudo que o sistema precisa fazer, como receber dados do componente, analisar critérios e gerar um nível de risco. Já nos não funcionais, foca-se em desempenho, simplicidade, segurança e facilidade de manutenção.

Com a finalidade de criar um protótipo funcional capaz de analisar componentes de IA de terceiros e gerar uma avaliação de risco automatizada.

5 Requisitos funcionais

REQUISITO FUNCIONAL:	DESCRIÇÃO:
RF01 - Cadastrar componente de IA	O sistema deve permitir que o usuário informe os dados de um componente de IA de terceiros, como nome, tipo, versão, origem, licença, data de atualização e outras características relevantes para a análise. Esse cadastro é a base de toda a avaliação de risco.
RF02 - Identificar o tipo de componente	O sistema deve classificar o item informado em uma das categorias previstas no projeto, como modelo de IA, "dataset", API de IA ou biblioteca/framework. Essa distinção é necessária porque cada tipo de componente apresenta riscos próprios e exige critérios específicos.
RF03 - Validar os dados de entrada	O sistema deve verificar se os dados enviados estão completos, coerentes e dentro do formato esperado. Esse requisito evita análises inválidas,

	reduz erros de processamento e melhora a confiabilidade do resultado.
RF04 - Analisar a procedência do componente	O sistema deve avaliar se o componente possui origem confiável, como repositório oficial, fornecedor identificado, documentação mínima e informações claras sobre o desenvolvedor ou mantenedor. A procedência influencia diretamente o nível de confiança do artefato.
RF05 - Verificar integridade do componente	O sistema deve analisar se há evidências de integridade, como “checksum”, “hash” ou qualquer assinatura digital, permitindo inferir se o componente pode ser validado quanto à autenticidade e à ausência de alterações não autorizadas.
RF06 - Avaliar licença de uso	O sistema deve identificar se o componente apresenta licença informada e, quando possível, se a licença é compatível com o uso proposto. A ausência de licença ou a existência de restrições relevantes deve impactar a pontuação de risco.
RF07 - Avaliar atualização e manutenção	O sistema deve verificar se o componente possui sinais mínimos de manutenção, como versão definida, data recente de atualização ou evidência de suporte contínuo. Componentes abandonados ou muito desatualizados devem receber maior peso de risco.
RF08 - Avaliar dependências associadas	O sistema deve considerar se o componente depende de outros pacotes, bibliotecas ou recursos potencialmente obsoletos, inseguros ou excessivos. Dependências aumentam a superfície de risco e devem compor a análise.
RF09 - Avaliar documentação de IA	O sistema deve verificar a existência de documentação específica relacionada ao contexto de IA, como “model card”, “dataset card”,

	descrição de limitações, finalidade de uso e informações mínimas sobre origem dos dados ou comportamento esperado.
RF10 - Considerar contexto crítico de uso	O sistema deve permitir que o usuário informe se o componente será utilizado em um contexto crítico, como segurança pública, saúde, decisão automatizada ou ambiente sensível. Nesses casos, o peso do risco deve ser ampliado.
RF11 - Calcular pontuação de risco	O sistema deve transformar os critérios avaliados em uma pontuação numérica, permitindo mensurar de forma padronizada o nível de risco do componente analisado.
RF12 - Classificar o risco	O sistema deve converter a pontuação final em categorias compreensíveis, como risco baixo, médio ou alto, facilitando a interpretação do resultado pelo usuário.
RF13 - Justificar a classificação	O sistema deve apresentar explicações textuais que mostrem quais fatores contribuíram para o resultado final. Esse requisito é essencial para garantir transparência, capacidade de interpretação e utilidade prática da ferramenta.
RF14 - Gerar recomendação final	O sistema deve informar uma recomendação objetiva ao usuário, como aprovar, revisar ou bloquear o componente, com base no nível de risco identificado.
RF15 - Exibir resultado da análise	O sistema deve retornar, ao final do processo, uma resposta organizada contendo os dados principais da análise, score, classificação, justificativas e recomendação.
RF16 - Permitir reanálise	O sistema deve possibilitar ao usuário ajustar informações do componente e realizar nova análise, simulando cenários diferentes ou corrigindo dados anteriormente inseridos.
RF17 - Registrar histórico básico das análises	O sistema pode armazenar localmente as análises realizadas para consulta posterior, permitindo comparar resultados e apoiar futuras evoluções do projeto.

RF18 - Disponibilizar API de análise	O sistema deve oferecer uma API para receber os dados do componente e retornar o resultado da avaliação, permitindo integração futura com outras aplicações.
---	--

6 Requisitos não funcionais

Requisito Não Funcional:	DESCRIÇÃO:
RNF01 - Usabilidade	O sistema deve possuir interface simples, objetiva e intuitiva, permitindo que o usuário compreenda facilmente como inserir os dados e interpretar o resultado da análise. Como se trata de um protótipo acadêmico, a clareza do fluxo é essencial para demonstrar valor.
RNF02 - Desempenho	O tempo de resposta da análise deve ser curto, de modo que o usuário obtenha retorno quase imediato após enviar os dados do componente. Como o motor inicial será baseado em regras, espera-se processamento leve e eficiente.
RNF03 - Confiabilidade	O sistema deve manter comportamento consistente ao analisar entradas semelhantes, evitando classificações aleatórias ou contraditórias. A lógica de risco precisa seguir critérios definidos e reproduzíveis.
RNF04 - Explicabilidade	O sistema deve fornecer justificativas compreensíveis sobre o resultado gerado. Não basta apenas classificar; é necessário mostrar por que o componente foi considerado de baixo, médio ou alto risco.
RNF05 - Segurança básica	O sistema deve validar entradas e tratar dados enviados de forma

	segura, prevenindo falhas simples de manipulação indevida, erros de formatação e comportamentos inesperados.
RNF06 - Modularidade	A aplicação deve ser organizada em módulos independentes, separando validação, regras, pontuação, classificação e retorno. Isso facilita manutenção, testes e futuras expansões.
RNF07 - Escalabilidade estrutural	O sistema deve ser pensado de modo que futuras camadas, como integração com banco de dados, análise automática externa ou classificador preditivo, possam ser adicionadas sem reescrever toda a solução.
RNF08 - Manutenibilidade	As regras de risco devem ser fáceis de atualizar. Isso é importante porque os critérios de segurança em IA evoluem rapidamente e o sistema precisa aceitar ajustes sem grande esforço técnico.
RNF09 - Portabilidade	O sistema deve poder ser executado localmente em ambiente acadêmico comum, sem depender de infraestrutura complexa ou serviços pagos obrigatórios.
RNF10 - Testabilidade	A arquitetura deve favorecer a realização de testes funcionais e validações de resultado, permitindo verificar se a classificação corresponde ao esperado em cenários simulados.
RNF11 - Interoperabilidade	A solução deve ser capaz de trocar informações por meio de API, facilitando integração com outras aplicações ou futuras interfaces web.
RNF12 - Clareza visual	O sistema deve apresentar resultados organizados e fáceis de ler, com distinção entre score, nível de risco, justificativas e recomendação final.
RNF13 - Disponibilidade local e conectividade	O sistema deve poder ser executado localmente em ambiente acadêmico comum, permitindo demonstração da interface, navegação entre telas, validação dos dados de entrada,

	geração de histórico e funcionamento do backend sem infraestrutura complexa. Entretanto, para a verificação externa completa das informações fornecidas pelo usuário, o sistema depende de conexão com a internet, pois consulta fontes oficiais como PyPI, Hugging Face Hub, GitHub e catálogo interno de APIs. Assim, a aplicação é executável localmente, mas sua verificação automática completa depende de conectividade.
RNF14 - Precisão mínima de classificação	O sistema deve manter coerência lógica entre critérios avaliados e resultado obtido, evitando decisões incoerentes.
RNF15 - Evolução tecnológica	O projeto deve permitir futura incorporação de bibliotecas adicionais, como “scikit-learn”, sem comprometer o funcionamento do núcleo inicial implementado com “FastAPI” e “Pydantic”.

7 Escopo do Projeto

Fase implementada:

Na versão atual do projeto, o sistema já foi desenvolvido como uma aplicação funcional com backend em Python e FastAPI, interface web integrada e persistência local de histórico. O objetivo deixou de ser apenas conceitual e passou a ser a entrega de um protótipo executável, capaz de receber dados de componentes de IA de terceiros, validar essas informações, consultar fontes externas oficiais, bloquear a análise quando houver divergências críticas e, quando não há erros, calcular o risco e apresentar o resultado ao usuário.

Funcionalidades implementadas:

- Recebimento de dados do componente por formulário web.

- Classificação por tipo de componente, diferenciando biblioteca/framework, modelo de IA, dataset, repositório e API de IA.
- Validação sintática dos dados de entrada no frontend e no backend.
- Verificação externa automática com base em fontes oficiais, conforme o tipo do componente informado.
- Consulta ao PyPI para bibliotecas Python.
- Consulta ao Hugging Face Hub para modelos e datasets.
- Consulta ao GitHub para repositórios.
- Verificação de APIs de IA com base em catálogo interno permitido.
- Comparação entre os dados informados pelo usuário e os dados retornados pela fonte oficial.
- Bloqueio da análise quando houver erro crítico, como nome inexistente, versão inexistente, origem incompatível ou licença divergente.
- Exibição progressiva da verificação na segunda etapa da interface, com animação visual e status por item verificado.
- Geração de score de risco apenas quando a verificação externa é aprovada.
- Classificação do risco em níveis, com justificativa e recomendação final.
- Exportação do resultado em formato JSON.
- Registro do histórico das análises em banco de dados local SQLite.
- Disponibilização de API REST para análise e integração futura.
- Testes automatizados para validação das regras, verificadores e fluxo do sistema.

Limitações atuais:

- O sistema não realiza auditoria profunda de vulnerabilidades em código-fonte ou binários.
- O sistema não substitui avaliação humana especializada em cenários críticos.
- A verificação externa depende da disponibilidade das fontes consultadas e de conexão com a internet.
- Alguns metadados públicos podem apresentar inconsistências entre fontes oficiais.
- Em certos casos, nem toda informação pode ser comprovada automaticamente, como contexto crítico de uso e algumas dependências indiretas.
- O sistema atua como apoio à decisão, não como mecanismo autônomo definitivo de aprovação de componentes.

Bibliotecas e tecnologias utilizadas:

- **FastAPI:** Permite construir uma API de forma rápida, moderna e fácil de testar. Utilizada para construção da API REST e organização do backend.
- **Pydantic:** Ajuda a validar e organizar os dados de entrada. Utilizada para validação e estruturação dos dados de entrada e saída.
- **Jinja2:** Utilizada para renderização das telas da interface web.
- **SQLite:** Utilizado para persistência local do histórico das análises.
- **Pytest:** Utilizado para testes automatizados do sistema.
- **Requests/HTTP client:** Utilizado para consultas às fontes externas oficiais.
- **Scikit-learn:** É biblioteca consolidada para criar classificadores simples, caso o projeto evolua para uma etapa preditiva. Usada para classificação complementar baseada em aprendizado de máquina. Permanece como

possibilidade de expansão futura para suporte preditivo, mas não compõe o núcleo da solução atual.

- **Pandas:** Lê arquivos CSV, Excel ou JSON, transforma esses dados em tabela, filtra informações, compara valores, conta ocorrências, limpa dados faltando, gera relatórios simples. Embora não seja essencial para o funcionamento do MVP, no futuro, ela pode ser usada para apoiar a estruturação dos dados e facilitar análises do sistema. Permanece como tecnologia opcional para futuras análises auxiliares e tratamento de dados, não sendo essencial ao funcionamento do protótipo atual.

8 Arquitetura e Modelagem

A arquitetura adotada no projeto é a **Arquitetura em Camadas (Layered Architecture)**, modular, implementada como aplicação web e API REST em Python com **FastAPI**. Essa escolha foi mantida porque representa de forma mais fiel o funcionamento real do sistema: receber os dados de um componente de IA de terceiros, validar essas informações, consultar fontes externas oficiais, verificar inconsistências críticas, aplicar regras de risco, calcular uma pontuação e devolver ao usuário uma resposta estruturada. Como se trata de um protótipo acadêmico funcional, essa arquitetura se mostrou mais adequada do que modelos mais complexos, pois mantém a solução organizada, compreensível e tecnicamente viável.

A escolha por arquitetura em camadas também se relaciona diretamente com os requisitos não funcionais definidos no projeto, especialmente modularidade, manutenibilidade, testabilidade, interoperabilidade e possibilidade de evolução futura. Em vez de concentrar toda a lógica em um único bloco, o sistema foi dividido em partes com responsabilidades específicas, permitindo maior clareza na implementação, melhor organização do fluxo e mais facilidade para manutenção e expansão.

Na versão atual, a arquitetura pode ser compreendida em **cinco camadas principais**. A primeira é a **camada de apresentação**, responsável pela interação com o usuário. Nela estão a interface web, as telas do sistema e os endpoints expostos ao cliente. Essa camada recebe os dados informados no formulário, exibe mensagens de validação, apresenta a verificação progressiva das informações e mostra o resultado final da análise.

A segunda é a **camada de aplicação**, responsável por coordenar o fluxo geral do sistema. Ela recebe a solicitação enviada pela interface, organiza a

sequência de execução, chama os validadores, aciona os verificadores externos, decide se a análise pode continuar ou se deve ser bloqueada e, quando permitido, encaminha os dados para cálculo de score e geração da resposta final. Essa camada funciona como elemento central de orquestração entre entrada, verificação, análise e persistência.

A terceira é a **camada de domínio**, considerada o núcleo lógico da solução. Nela estão as regras de negócio relacionadas à análise de risco. É nessa camada que o sistema avalia critérios como procedência, integridade, licença, atualização, dependências, documentação de IA e contexto crítico de uso. Também é nela que a pontuação é calculada, o risco é classificado em níveis e a justificativa textual é construída. Assim, a camada de domínio transforma informações verificadas em uma decisão estruturada e interpretável.

A quarta é a **camada de integração externa**, que passou a ser parte essencial do sistema na versão implementada. Nessa camada estão os módulos responsáveis por consultar fontes oficiais conforme o tipo do componente informado pelo usuário. Bibliotecas Python são verificadas no **PyPI**, modelos e datasets no **Hugging Face Hub**, repositórios no **GitHub** e APIs de IA em um catálogo interno permitido. Essa camada compara os dados informados no formulário com os metadados retornados pelas fontes oficiais e identifica divergências críticas, como nome inexistente, versão inválida, origem incompatível ou licença divergente. Quando ocorre erro bloqueante, a análise não avança para a etapa de score.

A quinta é a **camada de dados e persistência**, responsável pelo armazenamento local das análises realizadas. Nela é utilizado o **SQLite** para registrar histórico, resultados e dados associados às análises aprovadas. Essa camada permite rastreabilidade das execuções, reconsulta posterior e manutenção de um histórico funcional dentro da aplicação.

Embora o projeto não utilize um modelo MVC tradicional no sentido clássico de aplicações web completas, sua organização pode ser compreendida como compatível com a ideia de “MVC ou similar”, conforme solicitado no roteiro. O **Model** corresponde às estruturas de dados, entidades e regras de negócio do sistema. O **Controller** está representado pelas rotas e pela camada de aplicação, que recebem a requisição, coordenam validação, verificação e análise. Já a **View** aparece nas telas da interface, responsáveis por exibir o formulário, a verificação progressiva, o resultado da análise e o histórico. Dessa forma, a arquitetura atende à exigência acadêmica sem forçar uma classificação inadequada da solução principal.

Dentro dessa arquitetura, a modelagem do sistema foi organizada em módulos com funções específicas. O módulo de entrada recebe as informações do componente, como nome, versão, tipo, licença, origem, data de atualização e critérios adicionais. O módulo de validação verifica se os dados estão

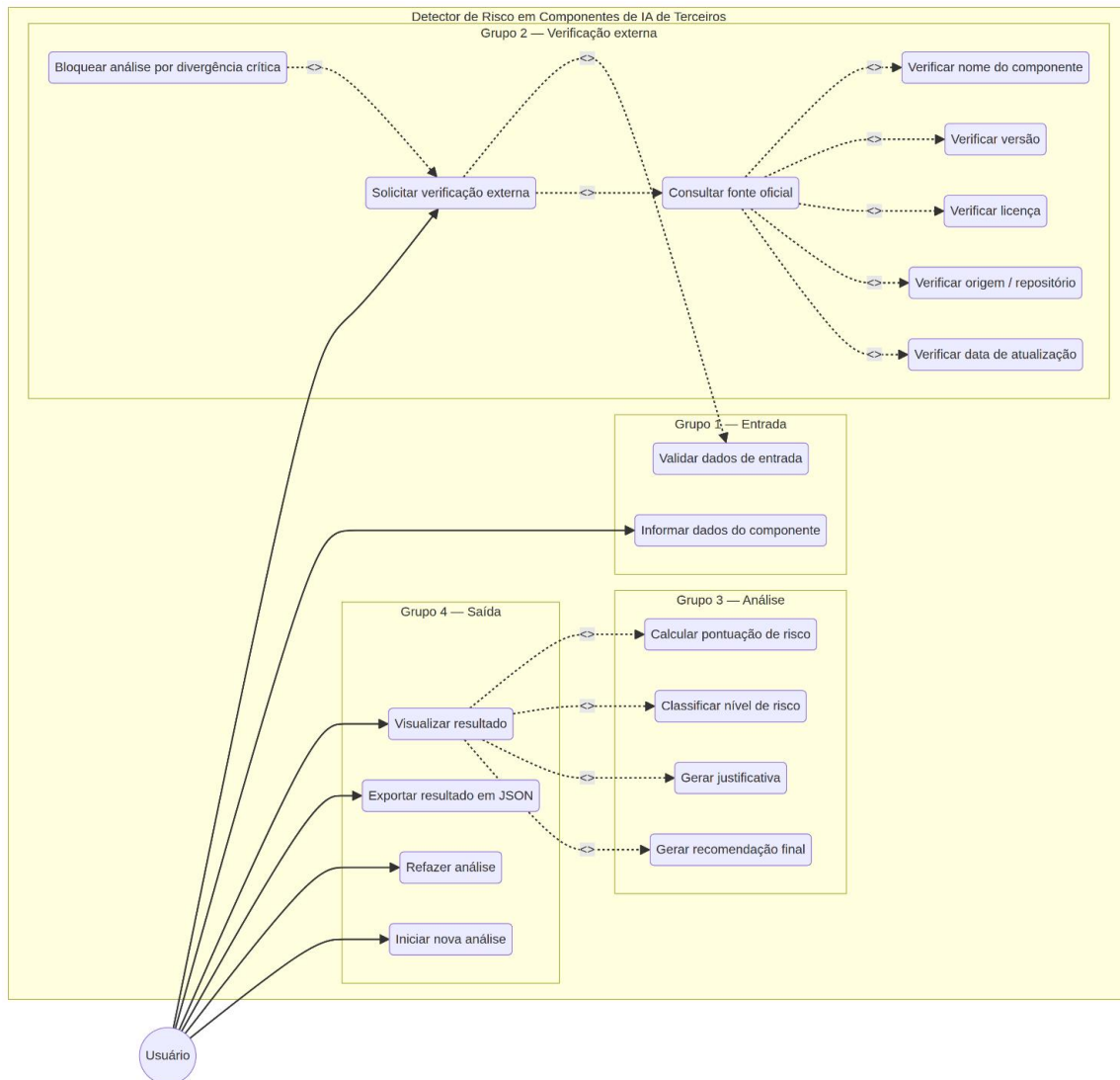
completos, coerentes e em formato adequado. Depois, o módulo de verificação externa consulta a fonte oficial correspondente ao tipo do componente e compara os dados informados pelo usuário com os dados retornados pela fonte. Caso haja divergência crítica, a análise é interrompida nessa etapa. Apenas quando a verificação é aprovada o sistema prossegue para o módulo de análise de risco, que aplica as regras previstas no projeto. Em seguida, o módulo de classificação transforma os critérios avaliados em score numérico e nível de risco. Por fim, o módulo de resposta devolve ao usuário uma saída estruturada com justificativa, recomendação, possibilidade de exportação em JSON e registro no histórico.

O fluxo dos dados ocorre de forma sequencial e controlada. Primeiro, o usuário preenche o formulário com os dados do componente. Em seguida, a interface envia essas informações ao backend. O sistema executa uma validação sintática inicial e, se ela for aprovada, aciona a verificação externa conforme o tipo do componente informado. Caso haja erro bloqueante, a continuidade é interrompida e o usuário é orientado a corrigir os dados. Caso a verificação seja bem-sucedida, o sistema continua para a análise de risco, calcula a pontuação, classifica o nível de risco, gera a justificativa, define a recomendação final e registra a análise no histórico. Esse fluxo torna a solução mais consistente com o problema do projeto, pois a decisão final não depende apenas daquilo que o usuário digitou, mas também de uma verificação automatizada em fontes externas oficiais.

Para representar essa modelagem na documentação e na apresentação, os diagramas UML mais adequados continuam sendo o **Diagrama de Casos de Uso** e o **Diagrama de Classes**. O diagrama de casos de uso deve mostrar o usuário informando os dados do componente, solicitando a verificação, visualizando o resultado, exportando a análise em JSON, realizando nova análise e, se desejado, consultando o histórico. Já o diagrama de classes deve representar as principais entidades e componentes da solução, incluindo classes como **ComponenteIA**, **ValidadorEntrada**, **VerificadorExterno**, **AnalizadorRisco**, **ClassificadorRisco**, **ResultadoAnalise** e **HistoricoAnalise**. Isso torna a representação mais fiel ao sistema implementado e mais coerente com o fluxo real da aplicação.

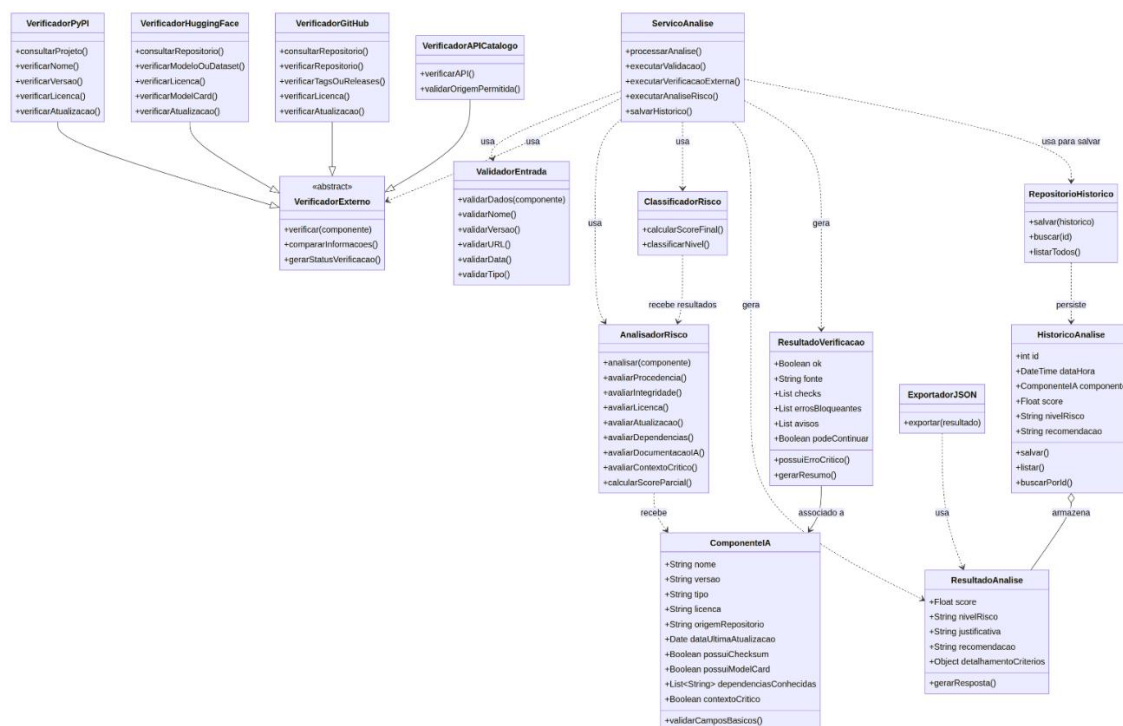
Assim, a arquitetura e a modelagem do projeto foram definidas com foco em clareza, organização, controle do fluxo e compatibilidade com o problema proposto. Em vez de permanecer apenas como um detector conceitual baseado em regras locais, a solução evoluiu para um protótipo funcional com interface integrada, verificação externa automatizada, análise explicável, persistência local e estrutura modular em camadas. Isso fortalece a coerência entre a proposta acadêmica, os requisitos definidos e a implementação efetivamente desenvolvida.

Diagramas:



Acima temos o **Diagrama de Casos de Uso**. Ele representa a interação do usuário com o sistema **Detector de Risco em Componentes de IA de Terceiros**, evidenciando o fluxo funcional da aplicação desde a entrada dos dados até a obtenção do resultado final. Inicialmente, o usuário informa os dados do componente que deseja avaliar, etapa a partir da qual o sistema executa a validação dos dados de entrada e solicita a verificação externa. Essa verificação consulta uma fonte oficial e realiza checagens específicas sobre nome do componente, versão, licença, origem ou repositório e data de atualização. Caso sejam identificadas divergências críticas, o sistema pode bloquear o prosseguimento da análise, impedindo a continuidade do fluxo até que os dados sejam corrigidos.

Após a verificação externa ser concluída com sucesso, o sistema prossegue para a etapa de análise, em que calcula a pontuação de risco, classifica o nível obtido, gera a justificativa e produz a recomendação final. Na etapa de saída, o usuário pode visualizar o resultado, exportá-lo em formato JSON, refazer a análise e iniciar uma nova avaliação. Dessa forma, o diagrama demonstra que a solução não se limita a gerar um score, mas organiza um fluxo completo de validação, verificação, análise e apresentação dos resultados, reforçando a coerência entre os requisitos do projeto e o comportamento real do sistema.



O **Diagrama de Classes** acima, representa a estrutura interna do sistema de forma modular, evidenciando a separação entre validação, verificação externa, análise de risco, classificação, exportação e persistência. No centro da solução está a classe **ServicoAnalise**, responsável por coordenar o fluxo principal da aplicação, acionando a validação dos dados, executando a verificação externa, encaminhando o componente para análise de risco e registrando o resultado no histórico. A classe **ComponenteIA** concentra os atributos essenciais do item analisado, como nome, versão, tipo, licença, origem, data de atualização e critérios adicionais, enquanto a classe **ValidadorEntrada** verifica a consistência e o formato das informações fornecidas pelo usuário.

A verificação externa é modelada por meio da classe abstrata **VerificadorExterno**, da qual derivam verificadores especializados para diferentes fontes, como **VerificadorPyPI**, **VerificadorHuggingFace**,

VerificadorGitHub e **VerificadorAPICatalogo**. Após essa etapa, a classe **AnalizadorRisco** avalia critérios como procedência, integridade, licença, atualização, dependências, documentação de IA e contexto crítico, e a classe **ClassificadorRisco** consolida o score final e define o nível de risco. O sistema trabalha ainda com dois resultados distintos: **ResultadoVerificacao**, que representa a saída da conferência externa, e **ResultadoAnalise**, que reúne score, nível de risco, justificativa, recomendação e detalhamento dos critérios avaliados. Complementando a estrutura, o diagrama inclui **ExportadorJSON**, responsável pela exportação do resultado, e o conjunto **RepositorioHistorico/HistoricoAnalise**, que representa a persistência local das análises realizadas. Assim, o gráfico demonstra de forma clara como os módulos se relacionam para compor o funcionamento completo da aplicação.

9 Evolução da Modelagem

A modelagem do projeto evoluiu conforme a ideia inicial foi sendo refinada e transformada em uma solução funcional. No começo, o sistema era pensado apenas como uma ferramenta simples para receber os dados de um componente de IA e gerar uma classificação de risco com base em regras locais. Essa estrutura atendia à proposta inicial, mas ainda era limitada para representar de forma adequada o problema de segurança no uso de componentes de IA de terceiros.

O primeiro avanço importante foi a substituição de uma lógica simples de aceitação ou rejeição por uma análise mais graduada, baseada em score, nível de risco e recomendação final. Isso tornou o sistema mais coerente com situações reais, nas quais um componente pode exigir revisão antes do uso, em vez de ser apenas aprovado ou bloqueado automaticamente.

Em seguida, a modelagem foi ampliada para considerar mais critérios de avaliação, como integridade, licença, atualização, dependências, documentação de IA e contexto crítico de uso. Essa mudança deixou o sistema mais completo e melhor alinhado aos requisitos funcionais definidos no projeto.

A principal evolução ocorreu com a inclusão da verificação externa. Antes, o sistema dependia basicamente das informações fornecidas pelo usuário. Com o refinamento da proposta, foi adicionada uma etapa de consulta a fontes oficiais, como PyPI, Hugging Face Hub, GitHub e catálogo interno de APIs. A partir disso, o fluxo passou a incluir uma checagem prévia antes do cálculo do risco. Quando há divergência crítica, a análise é interrompida e o usuário precisa corrigir os dados informados.

Essa mudança também exigiu ajuste na arquitetura, que passou a incluir um módulo específico de verificação externa, além dos módulos de entrada, validação, análise, classificação e persistência. Com isso, o sistema passou a refletir melhor o problema escolhido e a oferecer uma resposta mais confiável.

Outro refinamento importante foi a separação entre o resultado da verificação externa e o resultado final da análise de risco. Isso permitiu representar melhor o comportamento real da aplicação, em que primeiro se decide se o fluxo pode continuar e só depois se calcula a pontuação e o nível de risco.

Por fim, a modelagem também evoluiu ao incorporar histórico de análises, exportação em JSON e uma interface mais clara na etapa de verificação. Assim, o projeto deixou de ser apenas uma ideia conceitual e passou a ser uma solução funcional, modular e mais próxima de um cenário real de uso.

10 Metodologia

O desenvolvimento do projeto foi realizado em etapas sequenciais. Inicialmente, foi feito o levantamento do problema e a definição do escopo da solução, com foco na análise de risco de componentes de IA de terceiros. Em seguida, foram definidos os requisitos funcionais e não funcionais, que serviram como base para a modelagem da aplicação. Após isso, foi escolhida uma arquitetura em camadas, adequada ao fluxo do sistema e à separação das responsabilidades internas.

Na etapa de implementação, foram utilizadas bibliotecas compatíveis com a proposta do projeto, como FastAPI para o backend, Pydantic para validação dos dados, Jinja2 para a interface web e SQLite para persistência do histórico. Depois da implementação da estrutura principal, foram desenvolvidos os módulos de verificação externa, responsáveis por consultar fontes oficiais conforme o tipo do componente informado. Por fim, foram realizados testes funcionais e ajustes na interface e no fluxo da aplicação, com o objetivo de tornar o sistema mais consistente, explicável e alinhado ao problema proposto.

11 Implementação e Desenvolvimento Técnico

O sistema foi implementado como uma aplicação web com backend em Python, utilizando FastAPI para construção da API e organização das rotas. A

validação dos dados de entrada foi feita com Pydantic, permitindo verificar formato, consistência e obrigatoriedade dos campos informados pelo usuário. A interface foi integrada ao backend para permitir preenchimento do formulário, exibição da etapa de verificação, apresentação do resultado e reutilização do fluxo em novas análises.

No núcleo da aplicação, a solução foi organizada em módulos responsáveis por funções específicas. Há um módulo de validação de entrada, um módulo de verificação externa por fonte oficial, um módulo de análise de risco, um módulo de classificação e um módulo de persistência. A verificação externa foi implementada com consultas a diferentes fontes conforme o tipo do componente informado: PyPI para bibliotecas Python, Hugging Face Hub para modelos e datasets, GitHub para repositórios e catálogo interno para APIs de IA. Quando há divergência crítica entre os dados informados e os dados retornados pela fonte oficial, o sistema interrompe o fluxo antes do cálculo do score.

A persistência das análises foi implementada com SQLite, permitindo manter histórico local. Além disso, o sistema oferece exportação do resultado em JSON, o que amplia a reutilização das análises realizadas. Essa implementação tornou o projeto mais compatível com um cenário real de uso, pois a decisão final deixa de depender apenas das informações digitadas pelo usuário e passa a incluir uma etapa de conferência automatizada.

12 Testes e Validação

A validação do sistema foi realizada por meio de testes funcionais e cenários simulados, com o objetivo de verificar se o comportamento da aplicação estava coerente com os requisitos definidos no projeto. Foram testados casos em que os dados informados coincidiam com as informações das fontes oficiais, permitindo o avanço para a análise de risco, e também casos em que existiam divergências críticas, como nome inexistente, versão inválida, origem incompatível ou licença divergente, situações em que o sistema deveria bloquear a continuidade do fluxo.

Também foram realizados testes sobre o formulário de entrada, a navegação entre telas, o botão de limpeza dos campos, a exibição da verificação progressiva, a geração da pontuação de risco, a classificação final, a exportação em JSON e o registro do histórico em SQLite. Esses testes permitiram verificar se a aplicação respondia de forma consistente tanto em cenários válidos quanto em cenários com erro. Com isso, foi possível confirmar que o sistema atende à

proposta do projeto ao combinar validação de entrada, conferência externa, análise de risco e apresentação estruturada dos resultados.

13 Protótipo de Telas

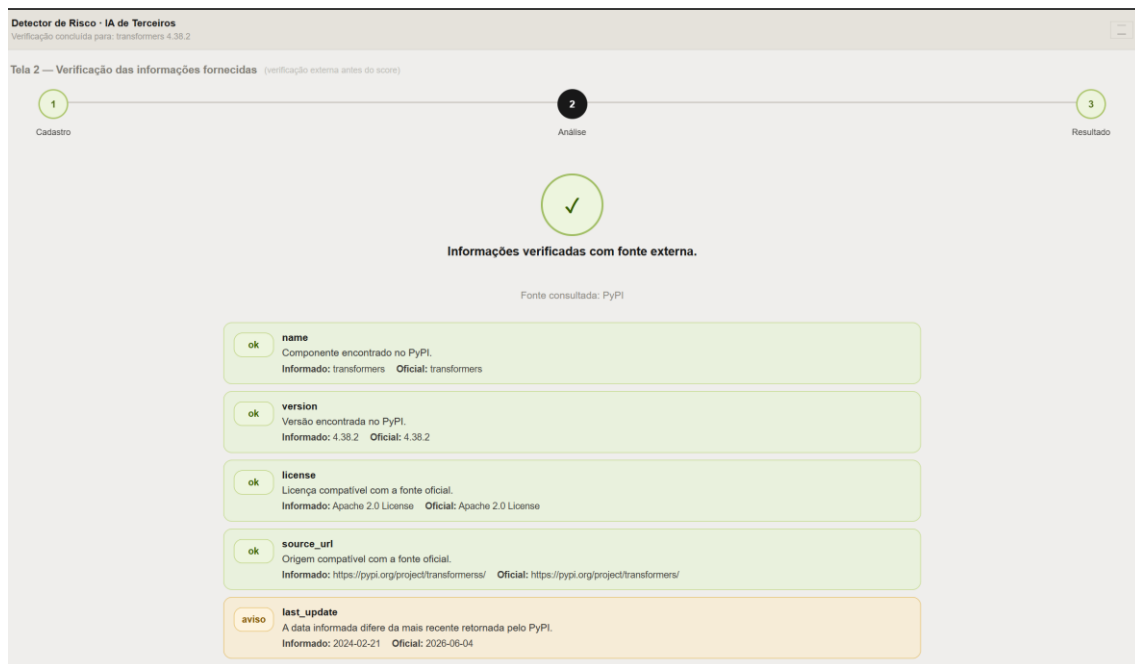
A navegação do sistema foi planejada para ser simples, sequencial e guiada por etapas, permitindo que o usuário acompanhe claramente cada fase do processo. Na versão atual apresentada no documento, o protótipo está organizado em três telas principais: cadastro, verificação e resultado.

O protótipo de tela, intitulado "Detector de Risco - IA de Terceiros", apresenta uma interface para o cadastro de novos componentes. No topo, há uma barra de progresso com três etapas: "1 Cadastro", "2 Análise" e "3 Resultado". O formulário principal é dividido em seções para coleta de dados:

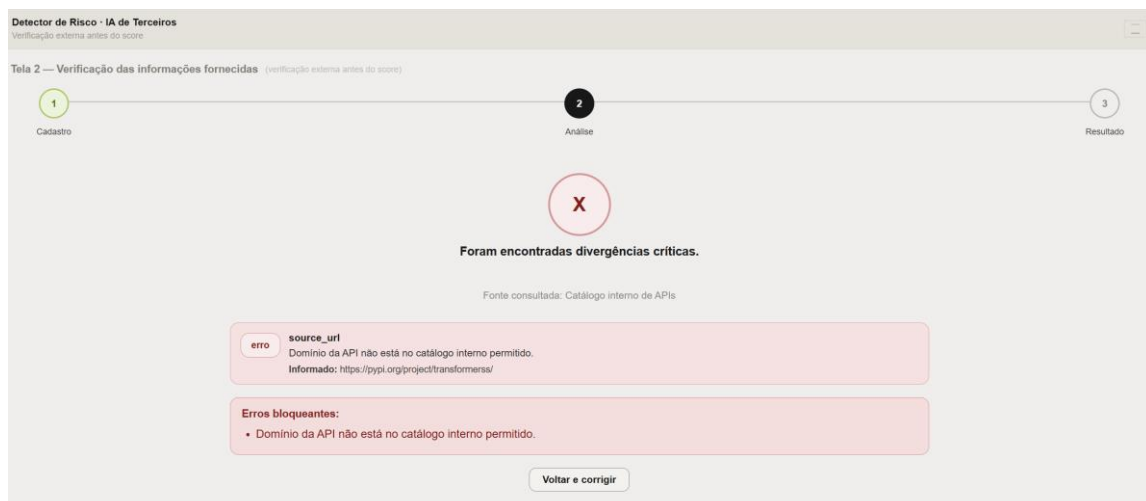
- NOME DO COMPONENTE:** Campo de texto com o exemplo "transformers".
- TIPO DE COMPONENTE:** Menu suspenso com a opção "Selecionar".
- ORIGEM / REPOSITÓRIO:** Campo de texto com o exemplo "https://exemplo.com".
- VERSÃO:** Campo de texto com o exemplo "4.38.2".
- LICENÇA:** Campo de texto com o exemplo "Apache 2.0".
- DATA DA ÚLTIMA ATUALIZAÇÃO:** Campo de texto com o formato "dd/mm/aaaa".
- CRITÉRIOS ADICIONAIS:** Seção com quatro opções de seleção:
 - ☐ Possui checksum / hash
 - ☐ Possui model card
 - ☐ Dependências conhecidas
 - ☐ Contexto crítico de uso

Na base da tela, há uma barra amarela com o aviso: "Aviso: Contexto crítico marcado amplia o peso da pontuação de risco." Abaixo disso, encontram-se dois botões: "Limpar" e "Analisar componente ->".

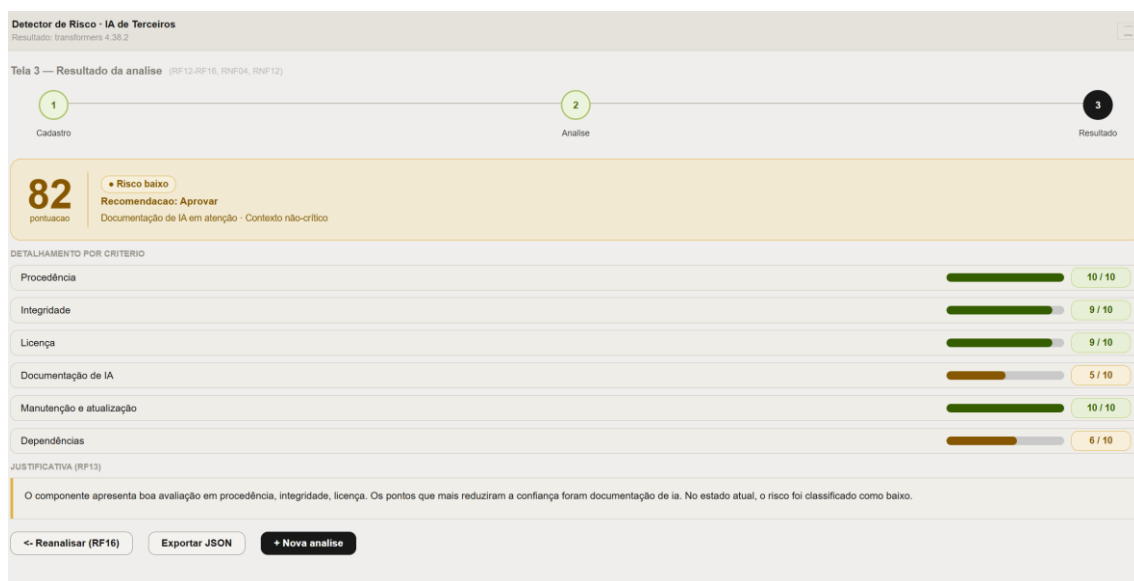
Na primeira tela, o usuário realiza o cadastro do componente de IA que deseja analisar. Nessa etapa, são informados dados como nome, versão, tipo do componente, licença, origem ou repositório, data da última atualização e critérios adicionais, como presença de checksum, existência de model card, dependências conhecidas e contexto crítico de uso. O formulário também oferece a opção de limpar os campos preenchidos ou prosseguir para a próxima etapa.



Na segunda tela, o sistema executa a verificação das informações fornecidas pelo usuário. Essa etapa funciona como uma validação progressiva, baseada em fontes externas oficiais, conforme o tipo do componente informado. Durante esse processo, a interface apresenta uma animação central de carregamento e exibe, item por item, o status de cada verificação realizada. Cada critério é mostrado visualmente em blocos com cores diferentes, indicando se a informação foi confirmada, se gerou aviso ou se apresentou erro crítico. Ao final da verificação, caso todos os critérios obrigatórios estejam corretos, o sistema libera o avanço para a próxima etapa.



Na terceira tela, caso seja encontrada alguma divergência crítica, a continuidade é bloqueada e o usuário recebe a opção de retornar para corrigir os dados informados.



Na quarta tela, o usuário visualiza o resultado final da análise de risco. Essa etapa só é liberada quando a verificação da tela anterior é concluída com sucesso. Nela são exibidos a pontuação final, a classificação do risco, a recomendação gerada pelo sistema e o detalhamento por critério avaliado. Além disso, o sistema apresenta justificativas textuais para explicar por que o componente foi classificado em determinado nível de risco, reforçando a transparência da solução. Nessa etapa, o usuário também pode optar por reanalisar o componente, iniciar uma nova análise ou exportar o resultado em formato JSON.

11 Próximos Passos

Com a implementação atual já concluída em nível funcional, os próximos passos do projeto passam a se concentrar no refinamento, ampliação e robustecimento da solução. A API em FastAPI, a validação dos dados com Pydantic, a interface web, o histórico em SQLite, a exportação JSON, os testes automatizados e a verificação externa por fonte oficial já foram implementados, o que permite o foco futuro em melhorias de cobertura, precisão e escalabilidade.

Uma das próximas evoluções previstas é o aprofundamento da análise técnica dos componentes. Atualmente, o sistema valida informações como nome, versão, licença, origem e atualização com base em fontes oficiais, mas pode evoluir para incluir verificações mais avançadas, como análise de dependências indiretas, conferência ampliada de artefatos publicados e consulta a bases públicas de vulnerabilidades.

Outro passo importante é o aumento da robustez operacional da verificação externa. Como a solução depende de fontes oficiais consultadas em tempo real, torna-se relevante adicionar estratégias de tolerância a falhas, cache local, tratamento mais detalhado para indisponibilidade temporária de serviços externos e padronização mais sofisticada de metadados divergentes entre plataformas.

No campo da usabilidade, o sistema pode evoluir para um painel administrativo mais completo, com usuário e login, com filtros no histórico, visualização estatística das análises realizadas, comparação entre componentes e detalhamento mais rico das justificativas. Isso tornaria a aplicação mais útil não apenas como protótipo acadêmico, mas também como ferramenta demonstrativa de apoio à decisão.

Também permanece como possibilidade futura a incorporação de técnicas preditivas e analíticas complementares, com bibliotecas como Scikit-learn e Pandas, desde que essa expansão mantenha a coerência com a proposta central do projeto. Essas tecnologias poderiam apoiar classificações auxiliares, análise de padrões históricos e relatórios mais elaborados, sem substituir o mecanismo principal de verificação e análise já implementado.

Por fim, o projeto pode ser expandido em contexto acadêmico para pesquisa, extensão ou integração com bases institucionais, especialmente na área de segurança aplicada a componentes de IA de terceiros. Dessa forma, a solução atual serve não apenas como entrega funcional da disciplina, mas também como base concreta para evolução futura.

14 Referências

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Artificial Intelligence Risk Management Framework (AI RMF 1.0)**. Gaithersburg: NIST, 2023. Disponível em: <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>. Acesso em: 10 abr. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **AI Risk Management Framework**. Gaithersburg: NIST, [s. d.]. Disponível em: <https://www.nist.gov/itl/ai-risk-management-framework>. Acesso em: 10 abr. 2026.

RAMÍREZ, Sebastián. **FastAPI**. [S. l.]: FastAPI, [s. d.]. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 10 abr. 2026.

RAMÍREZ, Sebastián. **First Steps**. In: **FastAPI Documentation**. [S. l.]: FastAPI, [s. d.]. Disponível em: <https://fastapi.tiangolo.com/tutorial/first-steps/>. Acesso em: 10 abr. 2026.

PYDANTIC SERVICES INC. **Pydantic Documentation**. [S. l.]: Pydantic, [s. d.]. Disponível em: <https://docs.pydantic.dev/latest/>. Acesso em: 10 abr. 2026.

SCIKIT-LEARN DEVELOPERS. **scikit-learn: machine learning in Python**. [S. l.]: scikit-learn, [s. d.]. Disponível em: <https://scikit-learn.org/>. Acesso em: 10 abr. 2026.

SCIKIT-LEARN DEVELOPERS. **Getting Started**. In: **scikit-learn documentation**. [S. l.]: scikit-learn, [s. d.]. Disponível em: https://scikit-learn.org/stable/getting_started.html. Acesso em: 10 abr. 2026.